

Integration Testing Report

eMagiz Ticketing System — Team 04

1. Purpose and Scope

This report documents **integration testing** of the eMagiz Ticketing System. Integration tests verify that multiple components work together correctly:

- **REST API** (Jersey / Tomcat)
- **PostgreSQL** database
- **Authentication & authorization** (AuthFilter, RoleFilter, @RolesAllowed)
- **Audit logging** (actions persisted after API calls)

2. Testing Strategy

Integration testing follows a **scenario-based** approach:

1. Authenticate and obtain a bearer token.
 2. Call API endpoints in realistic order (create → read → update).
 3. Assert HTTP status codes and JSON response shape.
 4. Optionally verify side effects in the database or audit log.
-

3. Test Environment (Reproducible Setup)

3.1 Prerequisites

- Docker & Docker Compose
- Postman (Desktop or Web) **or** cURL
- Network access to `localhost`

3.2 Test accounts

Role	Username	Password
Customer	demous	demopass
Support	demosup	demopass
Consultant	democons	demopass

3.3 Postman environment variables

Create a Postman **Environment** named `eMagiz Local`:

Variable	Initial value	Description
baseUrl	http://localhost:8080/api	API root
customerToken	(empty)	Filled after customer login
supportToken	(empty)	Filled after support login
consultantToken	(empty)	Filled after consultant login
ticketId	(empty)	ID of ticket created during test run
customerId	(empty)	User ID from login response

3.5 Saving tokens automatically (Postman)

On each **Login** request, add this to the **Tests** tab:

```
const body = pm.response.json();
pm.environment.set("customerToken", body.token); // rename per role
pm.environment.set("customerId", body.userId);
```

Use supportToken / consultantToken for other roles.

4. Integration Test Suites

Suite A — Authentication

IT-A1: Successful login (all roles)

Step	Action
1	POST {{baseUrl}}/users/login
2	Body (JSON): {"username": "demous", "password": "demopass"}
3	Headers: Content-Type: application/json

Expected: 200 OK, JSON contains token, userId, username, role (CUSTOMER).

Repeat for demosup (SUPPORT) and democons (CONSULTANT).

IT-A2: Invalid credentials

Step	Action
1	POST {{baseUrl}}/users/login
2	Body: {"username": "demous", "password":

Step	Action
	"wrong"}

Expected: 401 Unauthorized, error message in JSON.

IT-A3: Missing token on protected route

Step	Action
1	GET {{baseUrl}}/tickets (no Authorization header)

Expected: 401 Unauthorized.

Suite B — User Management (Support only)

IT-B1: List all users

Step	Action
1	Login as demosup → save supportToken
2	GET {{baseUrl}}/users
3	Header: Authorization: Bearer {{supportToken}}

Expected: 200 OK, array of users with id, username, role, company.

IT-B2: Create user

Step	Action
1	POST {{baseUrl}}/users
2	Header: Authorization: Bearer {{supportToken}}
3	Body:

```
{
  "username": "testcustomer1",
  "email": "testcustomer1@student.nl",
  "password": "TestPass123!",
  "role": "CUSTOMER",
  "company": "Test BV"
}
```

Expected: 201 Created, returned user has assigned id and uppercase role.

IT-B3: Customer cannot list users

Step	Action
1	Login as demous
2	GET <code>{{baseUrl}}/users</code> with customer token

Expected: 403 Forbidden.

IT-B4: User directory (all authenticated roles)

Step	Action
1	GET <code>{{baseUrl}}/users/directory</code>
2	Header: Authorization: Bearer <code>{{customerToken}}</code>

Expected: 200 OK, array with id, username, role, company (no passwords).

Suite C — Ticket Creation

IT-C1: Customer creates incident ticket

Step	Action
1	Login as demous
2	POST <code>{{baseUrl}}/tickets</code>
3	Headers: Authorization: Bearer <code>{{customerToken}}</code> , Content-Type: application/json
4	Body:

```
{
  "title": "Integration test – API timeout",
  "description": "Production API returns 504 since 09:00.",
  "type": "INCIDENT",
  "priority": "HIGH"
}
```

Expected: 201 Created, status = OPEN, creatorId = logged-in user, type = INCIDENT.

Save id as `{{ticketId}}`.

IT-C2: Support creates ticket on behalf of customer

Step	Action
1	Login as demosup, note customer userId from GET /users
2	POST {{baseUrl}}/tickets with support token
3	Body includes "creatorId": <customerUserId>, "type": "RFC"

Expected: 201 Created, ticket linked to selected customer.

IT-C3: Support creates internal ticket

Step	Action
1	POST {{baseUrl}}/tickets as support
2	Body: "type": "INTERNAL", no creatorId

Expected: 201 Created, type = INTERNAL.

IT-C4: Customer cannot create internal ticket

Step	Action
1	POST {{baseUrl}}/tickets as customer with "type": "INTERNAL"

Expected: 400 Bad Request.

Suite D — Ticket Read & Authorization

IT-D1: Customer reads own tickets

Step	Action
1	GET {{baseUrl}}/tickets/client/{{customerId}}
2	Header: Authorization: Bearer {{customerToken}}

Expected: 200 OK, array of tickets where user is creator; **no INTERNAL** tickets.

IT-D2: Customer reads single ticket

Step	Action
1	GET {{baseUrl}}/tickets/{{ticketId}} with customer token

Expected: 200 OK if ticket belongs to customer; 403 otherwise.

IT-D3: Support lists all tickets

Step	Action
1	GET {{baseUrl}}/tickets with support token

Expected: 200 OK, includes all ticket types.

IT-D4: Consultant reads assigned tickets only

Step	Action
1	Assign ticket to consultant (Suite E)
2	GET {{baseUrl}}/tickets/assignee/<consultantUserId> with consultant token

Expected: 200 OK for own assignee ID; 403 if assignee ID belongs to another user.

Suite E — Ticket Lifecycle (Support → Consultant)

Execute in order on one ticket created in IT-C1.

IT-E1: Add to review

Step	Action
1	PATCH {{baseUrl}}/tickets/{{ticketId}}/status
2	Support token
3	Body: {"status": "IN_REVIEW"}

Expected: 200 OK.

IT-E2: Accept ticket

Step	Action
1	PATCH .../status with body {"status": "ACCEPTED"}

Expected: 200 OK.

IT-E3: Assign to consultant

Step	Action
1	PUT {{baseUrl}}/tickets/{{ticketId}}/assignee/<consultantUserId>
2	Support token only

Expected: 200 OK, ticket assigneeId updated.

IT-E4: Consultant closes ticket

Step	Action
1	PATCH .../status with consultant token
2	Body: {"status": "CLOSED"}

Expected: 200 OK.

IT-E5: Deny path (alternate flow)

On a **new** ticket: OPEN → IN_REVIEW → DENIED using status PATCH as support.

Expected: Each transition returns 200 OK; final status DENIED.

Suite F — Comments & Team Notes

IT-F1: Customer adds public comment

Step	Action
1	POST {{baseUrl}}/tickets/{{ticketId}}/comments
2	Customer token
3	Body: {"content": "Any update on this issue?", "internal": false}

Expected: 201 Created, isInternal = false.

IT-F2: Support adds team note

Step	Action
1	Same endpoint, support token
2	Body: {"content": "Escalated to integration team.", "internal": true}

Expected: 201 Created, isInternal = true.

IT-F3: Customer cannot post internal note

Step	Action
1	POST comment with "internal": true as customer

Expected: 403 Forbidden.

IT-F4: Customer does not see internal comments

Step	Action
1	GET {{baseUrl}}/tickets/{{ticketId}}/comments as customer

Expected: 200 OK, response contains only public comments.

IT-F5: Support sees all comments

Step	Action
1	Same GET as support

Expected: Both public and internal comments returned.

Suite G — Priority & Audit Log

IT-G1: Customer changes priority (OPEN only)

Step	Action
1	On ticket with status OPEN: PATCH {{baseUrl}}/tickets/{{ticketId}}/priority

Step	Action
2	Body: {"priority": "CRITICAL"}

Expected: 200 OK while OPEN; 403 when status is not OPEN.

IT-G2: Global audit log (support)

Step	Action
1	GET {{baseUrl}}/audit-logs with support token

Expected: 200 OK, entries include TICKET_CREATED, STATUS_*, TICKET_COMMENT_ADDED.

IT-G3: Scoped audit log (customer)

Step	Action
1	GET {{baseUrl}}/audit-logs/accessible with customer token

Expected: 200 OK, only logs for customer's own non-internal tickets.

IT-G4: Per-ticket audit trail

Step	Action
1	GET {{baseUrl}}/audit-logs/ticket/{{ticketId}} with appropriate role token

Expected: 200 OK if user may access ticket; 403 otherwise.

5. Test Results Summary

Suite	Tests	Passed	Failed	Notes
A — Authentication	3	3	0	All roles login correctly
B — User management	4	4	0	RBAC enforced
C — Ticket creation	4	4	0	Type/role rules validated
D — Ticket read	4	4	0	Customer/consultant scoping

Suite	Tests	Passed	Failed	Notes
				works
E — Lifecycle	5	5	0	Full workflow OPEN → CLOSED verified
F — Comments	5	5	0	Internal notes hidden from customers
G — Priority & audit	4	4	0	Audit entries created per action
Total	29	29	0	

Results recorded during Team 04 test sessions on the Docker deployment (June 2026).

6. Issues Found During Integration Testing

ID	Severity	Description	Resolution
INT-01	High	Customer GET /tickets/client/{id} returned 403 (missing role in @RolesAllowed)	Fixed: added CUSTOMER to allowed roles
INT-02	High	Consultant GET /tickets/assignee/{id} returned 403	Fixed: added CONSULTANT role
INT-03	Medium	Customer/consultant called support-only/audit-logs and /users from UI polling	Fixed: added /audit-logs/accessible and /users/directory

7. Conclusion

Integration testing confirmed that the eMagiz Ticketing System API correctly implements authentication, role-based access, ticket lifecycle, comments, and audit logging. All 29 documented integration scenarios passed after fixes.